UE_C++基础

学习了一段时间的U++，基于大钊老师的《InsideUE4》，加了一些便于自己理解的注释

UObject

在UE4中，几乎所有的对象都继承于UObject（跟Java,C#一样），UObject为它们提供了基础的**垃圾回收（GC），反射（Reflextion），元数据(MetaData)，序列化(Serialization)，编辑器可见（Editable），Class Default Object**等，相应的，就有各种”UClass”的派生们定义了属性和行为的数据。
在UE中，3D世界是由Actors构建起来的，而Actor又拥有各种Component，之后又有各种Controller可以控制Actor（Pawn）的行为。

代码块

```
                    UCLASS / UPROPERTY / UFUNCTION
                              ↓ UHT 提取元数据
                            反射系统
                              ↓
              蓝图   编辑器   GC   序列化   网络同步
```

元数据(MetaData)

元数据就是”关于代码的描述信息”，在 UE5 里具体体现为 **UCLASS UPROPERTY UFUNCTION** 宏里填的那些参数，引擎用它们实现蓝图通信、编辑器显示、网络同步、序列化等功能。**描述这个类/函数/属性”是什么、有什么特性”的附加信息**，在运行时可以被查询。

代码块

```
UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Combat")<br>float Health;
```

这里 Health 的值是数据，而：

- 它能在编辑器里编辑吗？
- 它能被蓝图读写吗？
- 它属于哪个分类？

这些描述性信息就是**元数据**，由 UHT 提取后存进反射系统。

UE5 反射系统就是一套元数据系统

代码块

```
UCLASS()                    // 这个类存在于反射系统中UFUNCTION(BlueprintCallable) // 这个函数可以被蓝图调用UPROPERTY(Replicated)        // 这个属性需要网络同步
```

这些宏本身不影响逻辑，只是在告诉引擎：”帮我记录这些信息，运行时我要查询它们”

反射（Reflextion）

反射是 UE5 几乎所有”引擎能自动帮你做的事”的底层基础——只要你加了那几个宏，引擎就能在运行时理解你的代码结构，从而驱动蓝图、编辑器、GC、存档、联网这一整套系统。

1. 蓝图与 C++ 通信

反射让蓝图能”看见”你的 C++ 类、函数、属性：

代码块

```
UFUNCTION(BlueprintCallable)<br>void TakeDamage(float Amount);  // 没有反射，蓝图根本不知道这个函数存在
```

2. 编辑器面板显示属性

Details 面板能显示和编辑你的变量，靠的就是反射：

代码块

```
UPROPERTY(EditAnywhere)<br>float Health;  // 引擎通过反射找到这个属性，才能在面板里画出输入框
```

3. 序列化（存档/读档）

引擎保存关卡、存档游戏时，通过反射知道一个对象有哪些属性需要保存：

没有反射 → 引擎不知道你的类有什么成员 → 无法自动序列化

有了反射 → 遍历所有 UPROPERTY → 逐一保存

4. 网络同步

代码块

```
UPROPERTY(Replicated)<br>float Health;  // 引擎通过反射知道这个属性需要同步给客户端
```

5. 垃圾回收（GC）

UE5 的 GC 系统通过反射扫描所有 UPROPERTY 指针，判断对象是否还被引用，没有反射就无法追踪对象生命周期。

6. 运行时按名字查找/调用

代码块

```
// 通过字符串在运行时动态调用函数UFunction* Func = MyObject->FindFunction("TakeDamage");MyObject->ProcessEvent(Func, &Params);
```

控制台命令、自动化测试、Cheat Manager 都依赖这个能力。

垃圾回收（GC）

GC需要解决的问题是：“这个对象还有人用吗？没人用就可以思考了”

代码块

```
UPROPERTY() // ← 关键，告诉反射系统"这个指针我要追踪" AActor* Target; GC 的工作流程变成： 1. 通过反射，找到所有对象的所有 UPROPERTY 指针 2. 顺着这些指针，标记"还被引用的对象" 3. 没被标记到的对象 → 没人用了 → 销毁 这就是经典的 **Mark & Sweep（标记清除）** 算法。 --- 所以两者的关系是 反射 → 知道对象里有哪些指针 ↓ GC → 顺着这些指针追踪引用关系 ↓ 判断 → 没有引用链到达的对象 = 垃圾 = 可以回收//
 没有 UPROPERTY，GC 看不见Target 指向的对象随时可能被回收  再访问它 → 野指针 → 崩溃
```

UE5的GC核心原则只有一条：
凡是UObject，创建用引擎API，存储加UPROPERTY，访问IsValid，一般不用delete——违反任意一条，迟早崩溃。

1.最基础：UPROPERTY保护指针

2.TWeakObjectPtr：想引用，但不阻止恢复 可以被回收，只是被观察

常用于：UI引用角色、观察者模式、存储引用——不想因为自己持有引用而意外阻止对象调研。

3.IsValid()：访问前先确认对象还活着

代码块

```
// UE5 中永远不要裸访问 UObject 指针if (IsValid(MyActor)){   MyActor->TakeDamage(10.f);}`IsValid()` 同时检查两件事：指针 != nullptr+对象没有被标记为PendingKill（等待销毁）
```

4.手动触发Destory()

5.NewObject/SpawnActor创建对象

6.容器里存UObject也要注意

代码块

```
//
 正确：数组加 UPROPERTY，GC 能扫描到里面的每个元素UPROPERTY()TArray MyActors;//
 危险：GC 看不见数组里的指针TArray MyActors;
```

序列化(Serialization)

序列化就是把**内存中的对象状态**转换成可存储/传输的字节流；反序列化则是反向过程。UE5 中有两套维度来理解它：**C++ 底层机制**和**反射驱动机制**。

> 把对象的状态转换成可以存储/传输的格式，以及反向还原

序列化： 对象（内存）→ 二进制/JSON → 存档/网络传输

反序列化： 二进制/JSON → 对象（内存）← 读档/网络接收

编辑器可见（Editable）

Class Default Object 类默认对象（CDO）

CDO 的作用：

1. 编辑器里显示属性默认值的来源
2. SpawnActor 时新对象从 CDO 拷贝初始值
3. 序列化时只存”和 CDO 不同的部分”，节省空间

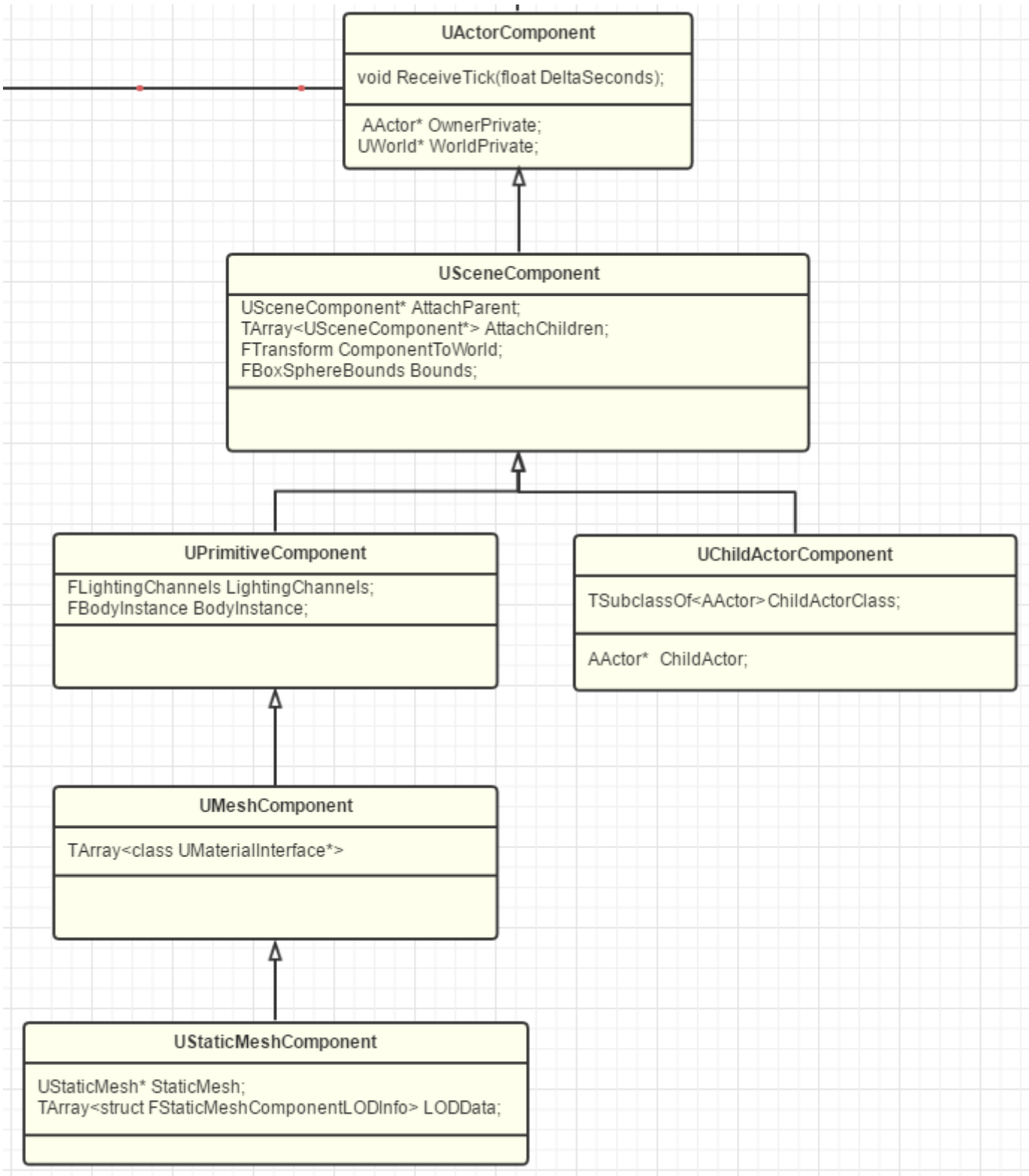
Actor

Replication（网络复制）,Spawn（生生死死），Tick(有了心跳)。

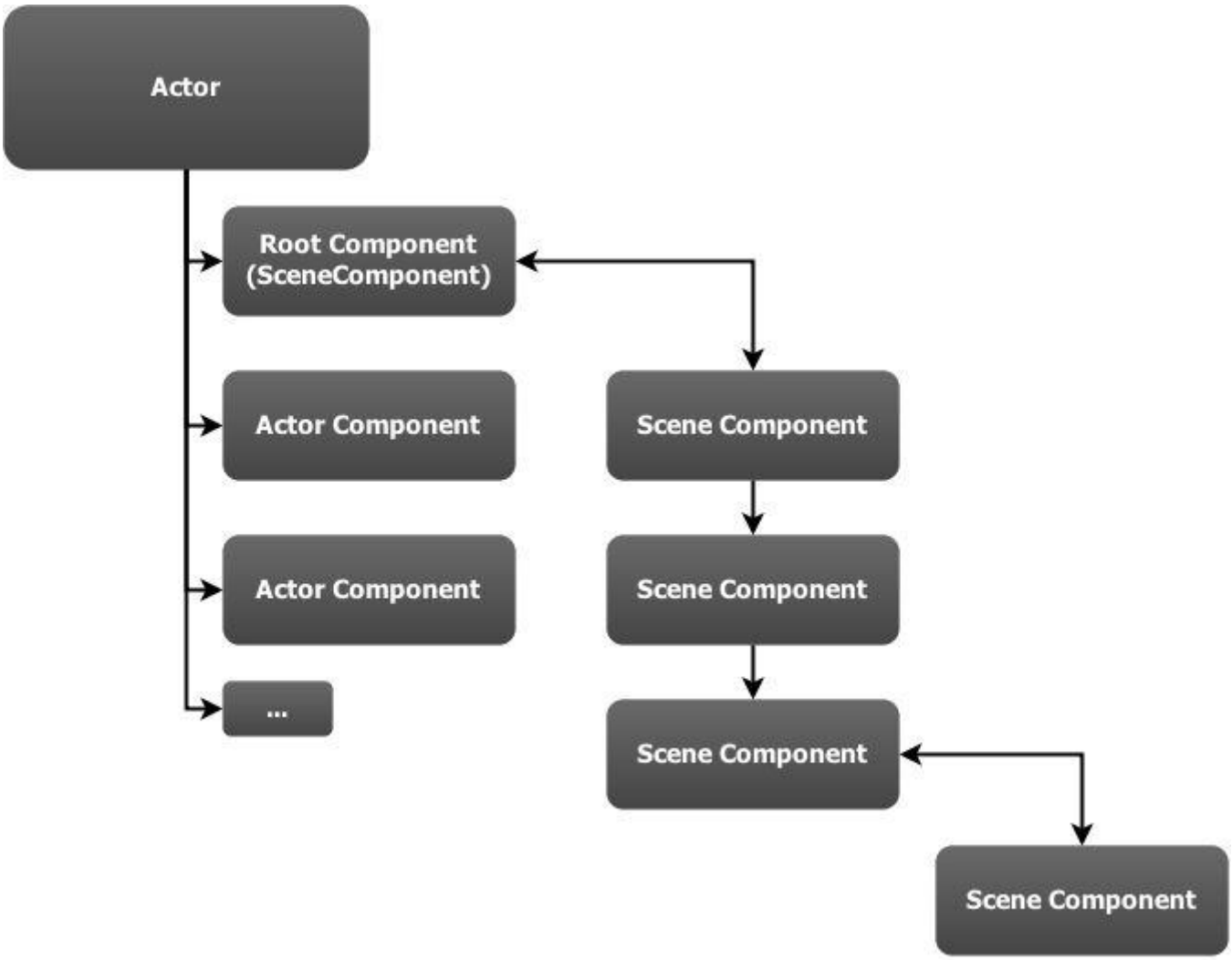
Actor无疑是UE中最重要的角色之一，组织庞大，最常见的有StaticMeshActor, CameraActor和PlayerStartActor等。Actor之间还可以互相“嵌套”，拥有相对的“父子”关系。

Component

UActorComponent也是基础于UObject的一个子类，这意味着其实Component也是有UObject的那些通用功能的。



ActorComponent下面最重要的一个Component就非SceneComponent莫属了。SceneComponent提供了两大能力：一是Transform，二是SceneComponent的互相嵌套



然后关于“而从游戏逻辑的实现来说，UE也是不推荐把**游戏逻辑**写在Component里面，所以你其实也没什么机会去写一个很复杂的Component.”这点，，我觉得老师应该是想说“不推荐把本**属于Actor的核心逻辑**塞进组件，但组件本身就是用来封装特定职责的逻辑的”；真正不推荐的是反过来控制Actor的核心行为，component不能作为Actor的上帝，只能专注于自己的职责。

代码块

```
Actor → 我是什么，我的核心身份和生命周期 Component → 我能做什么，某一块具体的功能模块 GameMode → 游戏规则是什么 GameState → 当前游戏状态是什么 PlayerState → 玩家数据是什么
```

不得不说UE的组件化设计很棒，就像衣服上的兜一样把东西分门别类装在一起，更显而易见。

Level

继承于UObject，作为Actor的容器，同时也划分了World，一方面支持了Level的动态加载，另一方面也允许了团队的实时协作

WorldSettings

一个World里有多个Level，这些Level在什么位置，是在一开始就加载进来，还是Streaming运行时加载。

World 是容器，Level 是内容。 平时感觉”一个关卡就是一个世界”，是因为大多数情况下 World 里只加载了一个 Level，所以感觉是同一个东西

WorldContext

一般就来说，对于独立运行的游戏，WorldContext只有唯一个。而对于编辑器模式，则是一个WorldContext给编辑器，一个WorldContext给PIE（Play In Editor）的World。WorldContext既负责World之间切换的上下文，也负责Level之间切换的操作信息。

代码块

```
namespace EWorldType{        enum Type        {                None,
     // An untyped world, in most cases this will be the vestigial worlds of
streamed in sub-levels                Game,                // The game world
（PIE运行时的游戏画面）                Editor,                // A world being
edited in the editor（整个Editor）                PIE,                // A Play
In Editor world(在Editor里面运行的游戏)                Preview,        // A
preview world for an editor tool                Inactive        // An editor
world that was loaded but not currently being edited in the level editor
 };}--UE5 同时存在两个并行的 World：按下播放之前└── Editor World（唯一的 World）
   在编辑器里看到、编辑的那个世界按下播放之后（PIE 启动）├── Editor World（仍然存在，
但暂时"冻住"）└── PIE World = Game World（新创建的模拟世界）        你绿框里看到的就是
这个PIE World 就是 Game World，是同一个 WorldContext，只是名字强调的角度不同：叫
PIE：强调它是在编辑器里运行的，不是真正打包发布的游戏叫 Game：强调它的 WorldContext 类
型是 EWorldType::PIE，行为和真实游戏一致
```

GameInstance

GameInstance里会保存着当前的WorldConext和其他整个游戏的信息，继承于UObject，所以就拥有了动态创建的能力，所以我们可以通过指定GameInstanceClass来让UE创建使用我们自定义的GameInstance子类。所以不论是C++还是BP，我们通常会继承于GameInstance，然后在里面编写应用于整个游戏范围的逻辑。

Engine

此处UEngine分化出了两个子类：UGameEngine和UEditorEngine。众所周知，UE的编辑器也是UE用自己的引擎渲染出来的，采用的也是Slate那套UI框架。好处有很多，比如跨平台比较统一，UI框架可以复用一套控件库，Dogfood等等。

UE会在不同模式下根据编译环境而采用不同的具体Engine类，而在基类UEngine里通过一个WorldList保存了所有的World。

- Standalone Game：会使用UGameEngine来创建出唯一的一个GameWorld，因为也只有一个，所以为了方便起见，就直接保存了GameInstance指针。
- 而对于编辑器来说，EditorWorld其实只是用来预览，所以并不拥有OwningGameInstance，而PlayWorld里的OwningGameInstance才是间接保存了GameInstance.

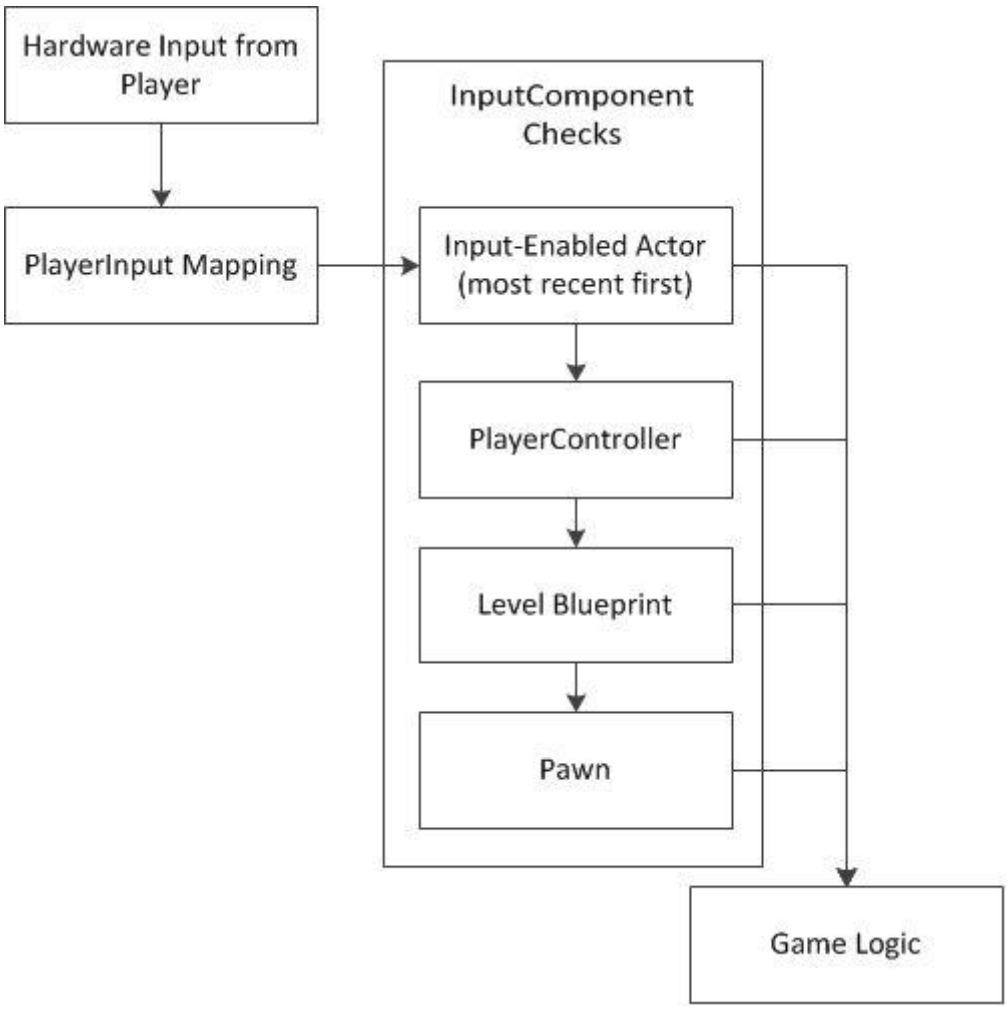
GamePlayStatics

相当于一个C++的静态类，只为蓝图暴露提供了一些静态方法。在想借鉴或者是查询某个功能的实现时，此处往往会是一个入口。

Pawn

从Actor中再派生出了APawn，并定义了3块基本的模板方法接口：

1. 可被Controller控制 响应输入和处理逻辑
2. PhysicsCollision表示 表达自身的存在
3. MovementInput的基本响应接口



不是”Pawn 给 Character 用”，而是**同一个 Controller 在不同时刻控制不同的 Pawn**，Character 只是其中一种 Pawn，载具是另一种，两者平级。这里容易产生游戏逻辑设计和基础概念混淆的错误。

DefaultPawn，SpectatorPawn，Character

DefaultPawn，默认带了一个DefaultPawnMovementComponent、spherical CollisionComponent和StaticMeshComponent。也是上述Pawn阐述过的三件套，只不过都是默认套餐。

派生于DefaultPawn的SpectatorPawn提供了一个基本的USpectatorPawnMovement（不带重力漫游），并关闭了StaticMesh的显示，碰撞也设置到了“Spectator”通道。观战的玩家们虽然也在当前地图里，但是我们并不需要真正的去表示它们，只要给他们一些摄像机“漫游”的能力。

UE就为我们直接提供了一个人形的Pawn来让我们操纵，也就是Character了。像人一样行走的CharacterMovementComponent， 尽量贴合的CapsuleComponent，再加上骨骼上蒙皮的网格。同样的三件套，不一样的配方。

Pawn因为是基础款，所以提供了最大的灵活性。

设计模式/架构模式

AController

我去这里写的很有意思。

1. **有Possess和UnPossess**，源码里也有PawnPendingDestroy等这些函数（未一一列出）；
2. **能够和Pawn对应起来**：**关联Pawn的能力**，能同时控制多个Pawn，当然，一个Pawn也可以被多个AController一起控制。
3. GameMode中也保存着AIControllerClass和PlayerControllerClass的配置，用于在适当的时候Spawn出Controller；
4. **持续的运行**：继承于Actor也就有了**EnableInput和Tick**；
5. Controller本身还可以**继续派生**下去（如AIController和PlayerController），也可以容纳Components；
6. 也带着一个SceneComponent所以可以摆放在世界中；自身也可以添加成员变量来记忆存储游戏状态；
7. **自身有状态**：自身也有一个FName StateName（Playing、Spectating、Inactive），**切换自身的状态**（运行，观察，非激活）；
8. 因为跟Pawn是平级的关系，只在运行的时候引用关联，所以对彼此独立存在不做强制约束，提高了灵活性。一个Pawn自身上也可以配置策略：
9. **可同步**：有能力在其他客户端上影分身，让他们都跟随我的步伐一致行动。
10. **可探查世界的对象**：基本的我得看得见知道当前世界里已经有哪些对象
11. **可在世界里移动**，我可以选择帐中千里之外遥控Pawn，也可以选择附身在一个Pawn身上，这样我才能多角度无死角的观察我可爱的Pawn们
12. **保存数据状态**，
13. **拥有一定的扩展继承组合能力**

14. **能够脱离Pawn存在**
15. **可挂载释放**
16. **操纵Pawn生死的能力**

Controller 和 Pawn 不是 1:1 绑定的，Controller 是持久的”意志”，Pawn 是可替换的”身体”——同一个 Controller 可以在不同时刻控制不同的 Pawn，Pawn 也可以没有 Controller。